Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #6
Searching and Sorting File data using callback

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 3

Unit Name: File Handling and Portable Programming

Topic: Searching and Sorting File data using callback

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

In continuation with our earlier discussion on searching and sorting, if searching and sorting needs to be done based on two different criteria, **a callback function can be used to define the searching and sorting logic dynamically**. This approach avoids the need to write separate sorting functions for each criterion.

Coding Example_1: Recursive binary search on an array having the integers from the file using callback and array of pointers.

```
#include<stdio.h>
#include<stdlib.h>

// added in this version
int is_even(int x)    // first criteria for searching, number must be even
{       return x%2 == 0;  }
int is_less_than_22(int x) // second criteria – number must be less than 22
{       return x<22;   }

int my_search(int a[],int low,int high,int key,int(*p)(int))
{
        // recursive solution
        int pos = -1;     int mid;
        if (low>high)
                return pos;
        else
        {
                mid = (low+high)/2;
                if(a[mid]==key && p(key))
                        pos = mid;
                else if(a[mid]>key)
                        return my_search(a,low,mid-1,key,p);
                else
                        return my_search(a,mid+1,high,key,p);
        }
        return pos;
}
int main()
{
        FILE *fp = fopen("numbers.txt", "r");
```

```
C:\Users\Dell>a
Enter the key to be searched in the file content
100
It is even and found at 9 position
not found

C:\Users\Dell>a
Enter the key to be searched in the file content
1000
not found
not found
```

```
    if(fp == NULL)
        printf("Cant access the file\n");
    else
    {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
        fclose(fp);

        int key;
        printf("Enter the key to be searched in the file content\n");
        scanf("%d",&key);
        // perform search on a collection of elements and print only if it is even
        int pos = my_search(p,0,n-1,key,is_even);
        //printf("result is %d",pos);
        if(pos !=-1)
            printf("It is even and found at %d position\n",pos);
        else
            printf("not found\n");

        // perform search on a collection of elements and print only if it is less than 22
        pos = my_search(p,0,n-1,key,is_less_than_22);
        //printf("result is %d",pos);
        if(pos !=-1)
            printf("It is less than 22 and found at %d position\n",pos);
        else
            printf("not found\n");
        fclose(fp);
    }

    return 0;
}
```

Coding Example_2: Menu based code to apply Selection sort on an array of pointers using callback and writing the sorted data to a new file. Open the sorted file and observe the contents

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
typedef struct Student
{       int roll;       char name[10];     }ST;
void init_ptr(ST *s,ST **sp,int n);
void display(ST *s, int n);
void display_ptr(ST **sp, int n);
void swap(ST **s1, ST **s2);
void sort(ST **sp,int n, int (*comp)(ST *, ST *));
void write_to_file(FILE *fpw, ST **sp,int n);

//New functions added now
int comp_roll(ST *s1, ST *s2)
{               return s1->roll > s2->roll;     }
int comp_name(ST *s1, ST *s2)
{               return strcmp(s1->name,s2->name)>0;     }

int main()
{
        FILE *fp = fopen("student_rollno_name.csv", "r");
        FILE *fpw = fopen("student_rollno_name_new.csv", "w");
        if(fp == NULL || fpw == NULL)
                printf("Cant access the file");


        else{
                int n = 0;  char line[2000];
                while(fgets(line,2000, fp) != NULL)
                        n++;
                //printf("Number of rows %d", n);
                rewind(fp);
                ST *s = malloc(n*sizeof(ST));
                fgets(line,2000, fp);
                int i = 0;
                while(fgets(line,2000, fp) != NULL)
                {
```

```
C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
1
Successfully written the sorted data to a file

C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
2
Successfully written the sorted data to a file

C:\Users\Dell\C1\unit4>a
press 1 to sort using roll_no
press 2 to sort using name
4
invalid input!!
```

```
                s[i].roll = atoi(strtok(line,","));
                strcpy(s[i].name,strtok(NULL,","));
                i++;
        }
        ST **sp = malloc(n*sizeof(ST*));
        init_ptr(s,sp,n);
        //display(s,n-1);
        //display_ptr(sp,n-1);
        printf("press 1 to sort using roll_no\n");
        printf("press 2 to sort using name\n");
        int choice;
        scanf("%d", &choice);
        switch(choice)
        {
                case 1:sort(sp,n-1, comp_roll); //display_ptr(sp,n-1);
                        break;
                case 2:sort(sp,n-1, comp_name); //display_ptr(sp,n-1);
                        break;
                default:printf("invalid input!!");exit(0);
        }
        write_to_file(fpw, sp,n-1);
        fclose(fpw);
    }
}
void sort(ST **sp,int n, int (*comp)(ST *, ST *))   // Function pointer in the parameter
{
    int i,j,pos;
    for(i = 0; i < n-1; i++)
    {
        pos = i;


        for(j = i+1; j<n;j++)
        {
            if(comp(sp[pos],sp[j]))
                pos = j;
        }
        if(pos != i)
            swap(&sp[pos], &sp[i]);
    }
}
```

```
void swap(ST **s1, ST **s2){
        ST *temp = *s1;         *s1 = *s2;        *s2 = temp;
}

void init_ptr(ST *s, ST **sp, int n){
        for(int i = 0 ; i < n;i++)
        {                   sp[i] = &s[i];   }
}

void display(ST *s, int n){
        for(int i = 0; i < n; i++)
                printf("%d %s\n", s[i].roll, s[i].name);
}

void display_ptr(ST **sp, int n){
        for(int i = 0; i < n; i++)
                printf("%d %s\n", sp[i]->roll, sp[i]->name);
}

void write_to_file(FILE *fpw, ST **sp,int n){
        for(int i = 0; i < n; i++)
                fprintf(fpw, "%d,%s", sp[i]->roll, sp[i]->name);
        printf("Successfully written the sorted data to a file\n");
}
```

One callback, endless Searching and Sorting possibilities!!